

📌 PHASE 1 — Backend Project Setup

Objective

Setup backend architecture and environment.

🔧 Technologies

- Python
 - Flask
 - MySQL
 - SQLAlchemy (optional ORM)
 - OpenPyXL
 - Pandas
 - APScheduler
-

✅ Tasks

Install Backend Libraries

Flask

flask-cors

mysql-connector-python

openpyxl

pandas

APScheduler

Create Backend Folder Structure

backend/

|

|—— app.py

|—— config.py

|—— routes/

|—— database/

├── models/
├── scheduler/
├── services/
└── exports/

Configure Flask Server

Features:

- API setup
 - CORS setup
 - JSON responses
 - error handling
-

Setup Environment Variables

Store:

- DB credentials
 - secret keys
 - email configs
-

Deliverables

- Flask server running
 - MySQL connection established
 - Base project structure ready
-

PHASE 2 — Database Design & Setup

Objective

Create and manage project database.

✔ Tasks

Create Database

Database Name

attendance_system

Create Tables

Students Table

Column	Type
id	INT
name	VARCHAR
roll_no	VARCHAR
department	VARCHAR
semester	VARCHAR
email	VARCHAR
face_encoding	TEXT
created_at	DATETIME

Attendance Table

Column	Type
id	INT
student_id	INT
date	DATE
time	TIME

Column	Type
status	VARCHAR

Admin Table

Column	Type
id	INT
username	VARCHAR
password	VARCHAR

Deliverables

- Database schema completed
 - Tables created
 - Relationships working
-

PHASE 3 — Authentication System

Objective

Create secure admin authentication.

Tasks

Admin Login API

Features:

- username/password validation
 - session/token creation
-

Authentication Middleware

Protect:

- dashboard routes
 - export routes
 - student management routes
-

Logout System

Destroy session/token.

 Deliverables

- Working authentication system
-

 PHASE 4 — Student Management APIs

Objective

Handle student registration and management.

 Tasks

Create APIs

Register Student API

Receives:

- student details
- face encoding

Stores data in database.

Fetch Students API

Returns:

- all students

- filters
 - pagination
-

Update Student API

Allows admin editing.

Delete Student API

Removes:

- student
 - attendance records
-

Deliverables

- Complete CRUD APIs
-

PHASE 5 — Attendance Management System

Objective

Handle attendance storage and validation.

Tasks

Attendance API

Receives:

- recognized student ID
- timestamp

Stores attendance.

Duplicate Attendance Prevention

Logic:

if already_marked_today:
 reject_attendance()

Very important feature.

Attendance Filtering APIs

Filter by:

- date
 - department
 - student
-

Attendance Status APIs

Returns:

- present students
 - absent students
 - total attendance
-

Deliverables

- Attendance system working
 - Duplicate prevention implemented
-

PHASE 6 — Excel Automation System

Objective

Automatically generate and update attendance Excel sheets.

Tasks

Create Monthly Excel Files

Example:

attendance_may_2026.xlsx

Update Excel Automatically

When attendance marked:

Backend:

- opens Excel file
- updates correct student row
- writes:

P (09:14 AM)

Auto Create Date Columns

If new day:

- create new column automatically
-

Generate Reports

Export:

- monthly attendance
 - filtered attendance
 - department reports
-

 Deliverables

- Excel automation fully working
-

 PHASE 7 — Auto-Absent Scheduler

Objective

Automatically mark absent students after college hours.

Tasks

Setup Scheduler

Using:

- APScheduler
-

Daily Automation Logic

At:

5:00 PM

System:

- checks students without attendance
 - marks them absent
 - updates Excel
-

Prevent Repeated Execution

Scheduler should:

- run once daily only
-

Deliverables

- Auto-absent system operational
-

PHASE 8 — Email Report System

Objective

Automatically send attendance reports.

✅ Tasks

Email Functionality

Send:

- daily report
- monthly report

To:

- admin/faculty email
-

Attach Excel File

System sends:

attendance_may_2026.xlsx

📦 Deliverables

- Email reporting system working
-

📌 PHASE 9 — Dashboard APIs

Objective

Provide analytics data to frontend dashboard.

✅ Tasks

Analytics APIs

Return:

- total students
- present count
- absent count
- attendance percentage

Graph APIs

Provide:

- weekly data
- monthly trends
- department analytics

Deliverables

- Dashboard APIs completed

PHASE 10 — Backend Optimization & Testing

Objective

Stabilize backend before final demo.

Tasks

Error Handling

Handle:

- invalid requests
- DB errors
- duplicate entries

API Testing

Test all APIs using:

- Postman

Performance Testing

Ensure:

- fast attendance marking
 - stable Excel updates
-

Logging System

Log:

- attendance activity
 - errors
 - unknown events
-

Deliverables

- Stable production-ready backend